SRM Institute of Science and Technology
College of Engineering and Technology
Department of Electronics and Communication Engineering

21ECC333L- CMOS ANALOG AND DIGITAL VLSI LABORATORY
Sixth Semester, 2025-26 (Even semester)
Mini Project Report

Name : **G. Yagneswar**

Register No. : **RA2311067010027**

Title of Mini Project : **Design of an I2C (Inter Integrated Circuit)**

Date of Submission :

Particulars	Max. Marks	Marks Obtained
Design	20	
Demo verification& viva	15	
Project Report	05	
Total	**40**	

REPORT VERIFICATION

Date :

Staff Name : Dr. J. Manjula

Signature :

TITLE

1. OBJECTIVE

The objective of this project is to design and implement an I2C (Inter-Integrated Circuit) Master Controller using Verilog HDL and to verify its functionality through simulation. The design focuses on generating proper I2C communication signals including START condition, address transmission, data transfer, acknowledgment (ACK/NACK), and STOP condition. The goal is to ensure reliable serial communication between master and slave devices while validating protocol compliance.

2. INTRODUCTION

I2C is a widely used synchronous, serial, half-duplex communication protocol developed by Philips Semiconductor (now NXP) that allows multiple master and slave devices to communicate over just two lines — a Serial Clock Line (SCL) and a Serial Data Line (SDA). It is commonly employed in embedded systems for interfacing peripherals such as sensors, EEPROMs, and display controllers, where simplicity and low pin count are priorities.

The protocol operates through a well-defined sequence: the master initiates a transaction by generating a START condition (SDA pulled low while SCL remains high), followed by transmitting a 7-bit slave address along with a read/write bit. The addressed slave responds with an Acknowledgement (ACK) by pulling SDA low during the 9th SCL clock pulse. If acknowledged, the master proceeds to transfer 8 bits of data, after which the slave again sends an ACK. The transaction is concluded with a STOP condition (SDA released high while SCL is high).

The design implemented here is a single-master I2C controller targeting write operations, with a finite state machine (FSM) governing all state transitions. An ACK error detection mechanism is incorporated to identify NACK responses from the slave, enabling robust error handling.

3. SOFTWARE

- Verilog HDL – RTL design
- Synopsys VCS – Simulation
- Synopsys Verdi – Waveform analysis and debugging
- Design Compiler (DC) – Synthesis
- IC Compiler II (ICC2) – Physical design

4. DESCRIPTION

The I2C Master Controller is implemented in Verilog HDL as a clocked finite state machine (FSM) operating on a 100 MHz system clock (period = 10 ns, toggled every 5 ns in the testbench). The design consists of two modules: the main controller i2c and the verification testbench i2c_tb.

State Machine Architecture

The FSM uses a 3-bit state register with seven explicitly defined states encoded as parameters:

State Encoding Purpose IDLE 3'd0 Waits for start signal START 3'd1 Generates I2C START condition ADDR 3'd2 Transmits 7-bit address + write bit ACK_ADDR 3'd3 Samples slave ACK after address DATA 3'd4 Transmits 8-bit data byte ACK_DATA 3'd5 Samples slave ACK after data STOP 3'd6 Generates I2C STOP condition The FSM is driven by a single always @(posedge clk or negedge rst_n) block, making it a fully synchronous design with asynchronous active-low reset (rst_n).

State-by-State Operation

IDLE: The controller deasserts busy, holds SCL high, releases SDA (sda_en = 0), and clears ack_error. Upon detecting start = 1, it loads the shift register with {addr, 1'b0} — the 7-bit address appended with a write bit of 0 — asserts busy, and transitions to START.

START: The master asserts sda_en = 1 and pulls sda_out low while SCL is still high, generating the mandatory I2C START condition (SDA falling edge while SCL is high). SCL is then immediately pulled low to begin the clocking sequence, and bit_cnt is preloaded to 4'd7 to count down 8 bits. The machine advances to ADDR.

ADDR: SCL is toggled on every clock cycle (scl <= ~scl). On the falling edge of SCL (!scl), the master enables SDA and places the current bit shift_reg[bit_cnt] on the line. On the rising edge of SCL (scl is high), it checks whether bit_cnt == 0. If so, SCL is forced low and the machine moves to ACK_ADDR; otherwise bit_cnt is decremented. This transmits all 8 bits of {addr, 1'b0} MSB-first.

ACK_ADDR: SCL continues toggling. The master releases SDA by deasserting sda_en, allowing the slave to drive the line. On the rising SCL edge, the master samples sda. If sda !== 1'b0 (SDA is not pulled low), a NACK is detected, ack_error is set to 1, and the machine jumps to STOP. If sda == 1'b0 (ACK received), the data byte is loaded into shift_reg, bit_cnt is reset to 4'd7, and the machine transitions to DATA.

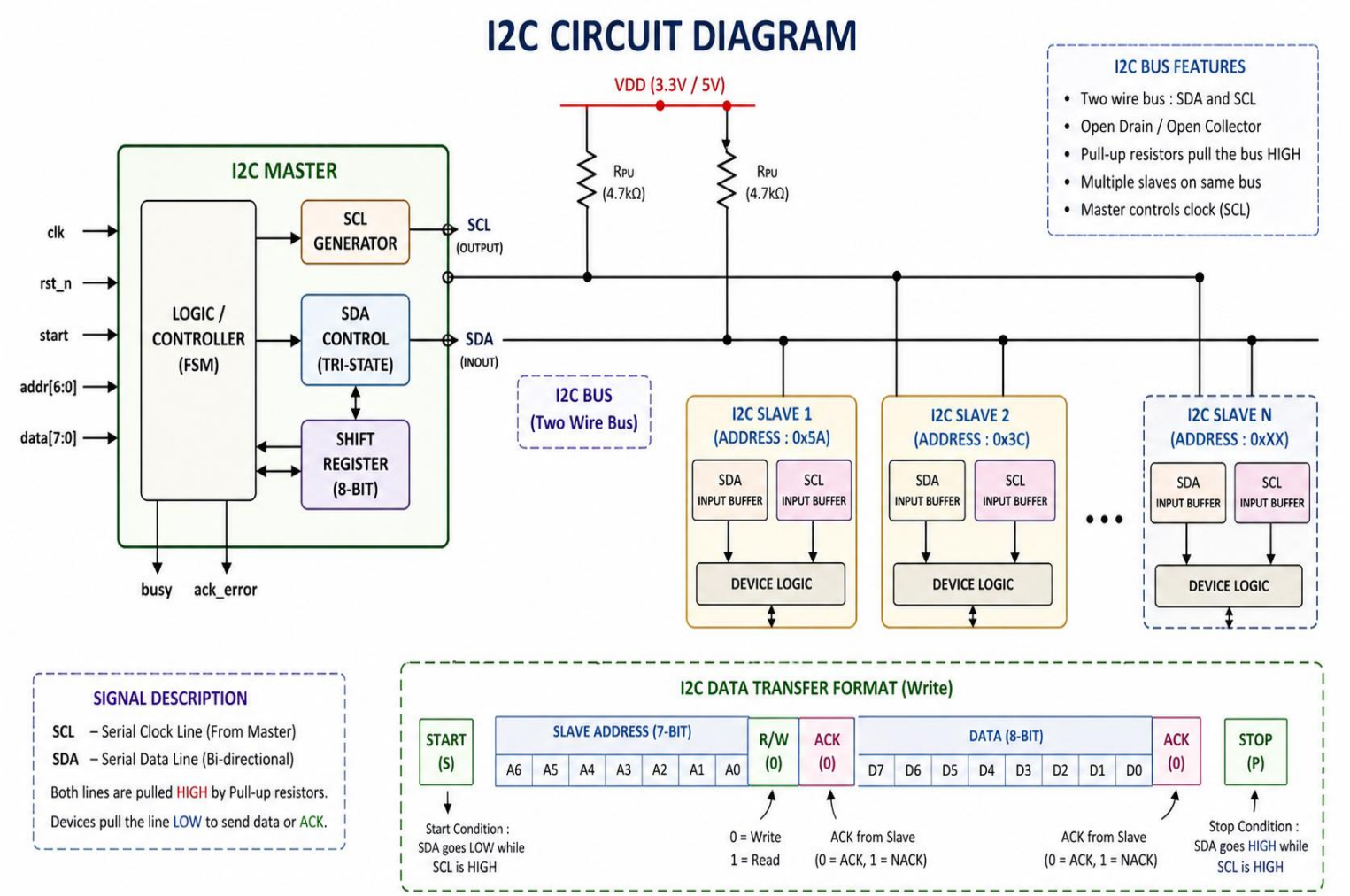
DATA: Identical in operation to ADDR. The 8-bit data byte stored in shift_reg is clocked out MSB-first on SDA, one bit per SCL cycle. When bit_cnt reaches zero, SCL is pulled low and the machine moves to ACK_DATA.

ACK_DATA: Identical in operation to ACK_ADDR. The master releases SDA and samples the slave's response on the rising SCL edge. If NACK is detected, ack_error is set. Unlike ACK_ADDR, the machine always proceeds to STOP regardless of the ACK outcome — the data transfer is complete either way.

STOP: SCL is released high (scl = 1), and SDA is initially held low (sda_out = 0) with sda_en = 1. Once SCL is confirmed high, sda_out is raised to 1, generating the I2C STOP condition (SDA rising while SCL is high). The machine then returns to IDLE.

5. DESIGN

The I2C Master Controller was designed using Verilog HDL with a finite state machine based architecture. The design is partitioned into two files: the RTL implementation (i2c.v) and the verification testbench (i2c_tb.v). The controller handles a complete I2C write transaction autonomously once triggered by the host, managing all protocol-level details including START condition, address phase, data phase, ACK sampling, and STOP condition internally.



On reset, all registers are initialized to safe default values: SCL is held high, SDA is released, busy is cleared, and the FSM returns to IDLE. This ensures predictable startup behavior.

The write transaction is initiated when the host asserts start for one clock cycle. At this point, the address and write bit are concatenated as {addr, 1'b0} and loaded into shift_reg. This combined 8-bit value is then serially shifted out MSB-first during the ADDR state by indexing shift_reg[bit_cnt] and decrementing bit_cnt on each SCL rising edge. The same shift-and-decrement mechanism is reused in the DATA state for the 8-bit data byte, keeping the datapath logic compact and consistent.

SCL toggling is implemented directly within the FSM using scl <= ~scl in both the ADDR and DATA states, producing a clock that toggles every system clock cycle. Bit output occurs on the falling SCL edge, and bit counting occurs on the rising SCL edge, which correctly satisfies the I2C requirement that data must be stable while SCL is high.

ACK sampling in both ACK_ADDR and ACK_DATA states is performed by reading the sda inout wire directly while sda_en is deasserted. The use of the !== operator (case inequality) rather than != ensures that high-impedance (1'bz) and unknown (1'bx) values on SDA are correctly treated as NACK, adding robustness to the detection logic. This is a deliberate design choice to prevent false ACK detection during undriven or contended bus conditions.

CODE :

```
`timescale 1ns/1ns

module i2c (

    input wire clk,

    input wire rst_n,

    input wire start,

    input wire [6:0] addr,

    input wire [7:0] data,

    output reg busy,

    output reg scl,

    inout wire sda

);

    // State Encoding

    parameter IDLE = 3'd0;

    parameter START = 3'd1;

    parameter ADDR = 3'd2;

    parameter ACK_ADDR = 3'd3; //  NEW: ACK after Address

    parameter DATA = 3'd4;

    parameter ACK_DATA = 3'd5; //  NEW: ACK after Data

    parameter STOP = 3'd6;

    reg [2:0] state;
```

```
reg [3:0] bit_cnt;

reg [7:0] shift_reg;

reg sda_out;

reg sda_en;

reg ack_error; //  NEW: ACK error flag


// SDA Tri-state Buffer

assign sda = sda_en ? sda_out : 1'bz;

always @(posedge clk or negedge rst_n) begin

    if (!rst_n) begin

        state <= IDLE;

        scl <= 1'b1;

        sda_en <= 1'b0;

        sda_out <= 1'b1;

        busy <= 1'b0;

        bit_cnt <= 4'd0;

        shift_reg <= 8'd0;

        ack_error <= 1'b0; //  NEW

    end else begin

        case (state)

            IDLE: begin

                busy <= 1'b0;

                scl <= 1'b1;

                sda_en <= 1'b0;

                ack_error <= 1'b0;

                if (start) begin

                    busy <= 1'b1;
```

```
        shift_reg <= {addr, 1'b0}; // Address + Write bit
        state <= START;
    end
end
START: begin
    sda_en <= 1'b1;
    sda_out <= 1'b0; // Pull SDA low while SCL high
    scl <= 1'b0; // Pull SCL low to begin
    state <= ADDR;
    bit_cnt <= 4'd7;
end

ADDR: begin
    scl <= ~scl;
    if (!scl) begin
        sda_en <= 1'b1;
        sda_out <= shift_reg[bit_cnt];
    end else begin
        if (bit_cnt == 0) begin
            state <= ACK_ADDR; //  Go to ACK state
            scl <= 1'b0;
        end else begin
            bit_cnt <= bit_cnt - 1;
        end
    end
end
```

```
//  NEW: ACK After Address
ACK_ADDR: begin
   scl <= ~scl;
   sda_en <= 1'b0; // Release SDA — Slave will drive it
   if (scl) begin // Sample on rising edge
      if (sda !== 1'b0) begin
         ack_error <= 1'b1; // NACK received
         state <= STOP;
      end else begin
         // ACK received  — proceed to DATA
         shift_reg <= data;
         bit_cnt <= 4'd7;
         state <= DATA;
      end
   end
end

DATA: begin
   scl <= ~scl;
   if (!scl) begin
      sda_en <= 1'b1;
      sda_out <= shift_reg[bit_cnt];
   end else begin
      if (bit_cnt == 0) begin
         state <= ACK_DATA; //  Go to ACK state
         scl <= 1'b0;
      end else begin
```

```
            bit_cnt <= bit_cnt - 1;
        end
    end
end

// NEW: ACK After Data
ACK_DATA: begin
    scl <= ~scl;
    sda_en <= 1'b0; // Release SDA — Slave will drive it
    if (scl) begin // Sample on rising edge
        if (sda !== 1'b0) begin
            ack_error <= 1'b1; // NACK received
        end
        state <= STOP; // Proceed to STOP either way
    end
end
STOP: begin
    scl <= 1'b1;
    sda_en <= 1'b1;
    sda_out <= 1'b0;
    if (scl) begin
        sda_out <= 1'b1; // SDA rises while SCL high = STOP
        state <= IDLE;
    end
end
default: state <= IDLE;
endcase
```

```
        end
    end
endmodule
```

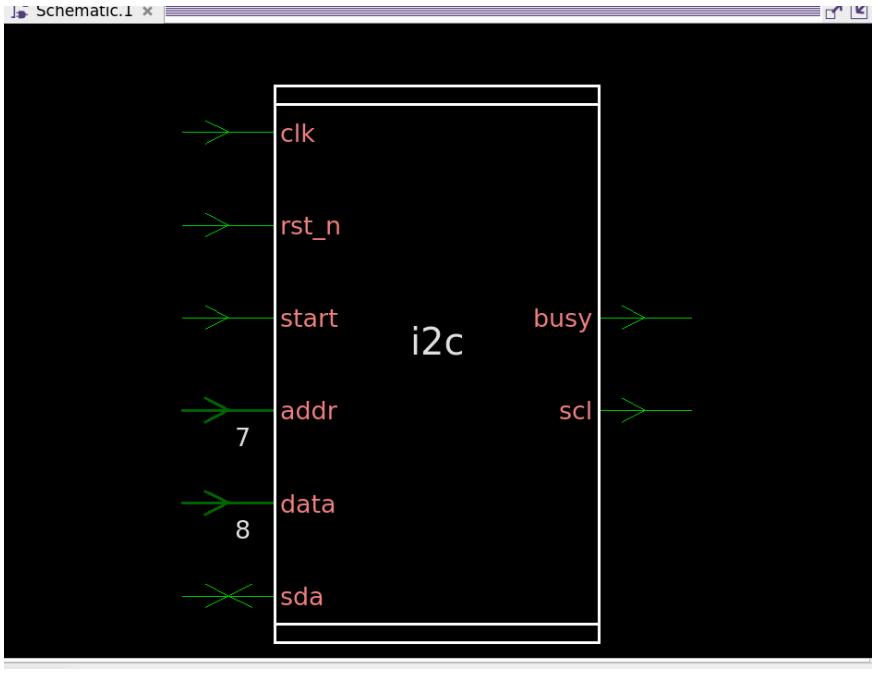
TESTBENCH :

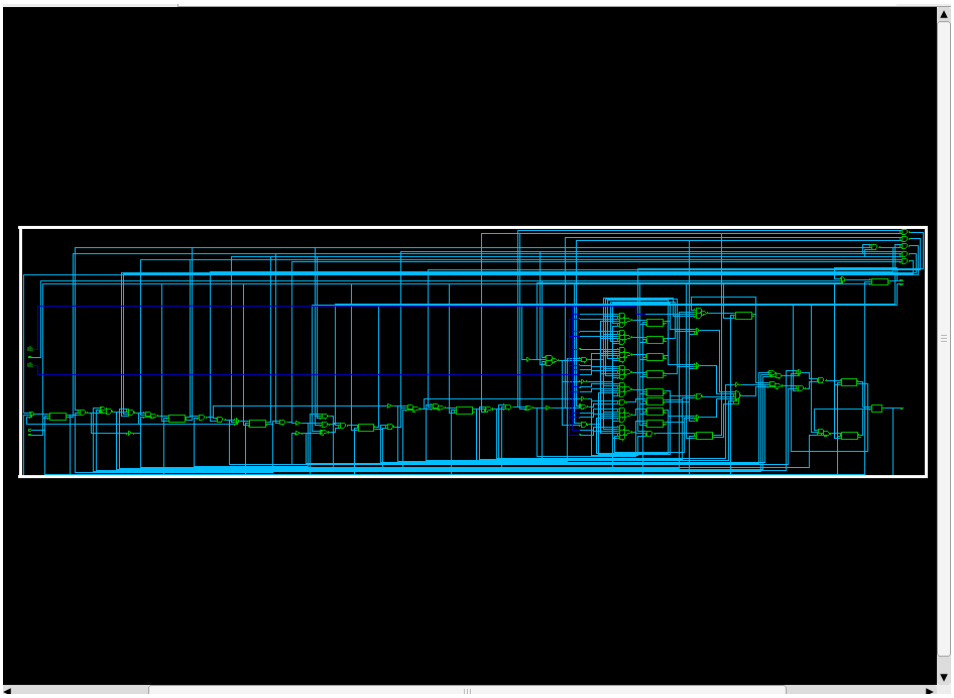
```
`timescale 1ns/1ns
`include "i2c.v"
module i2c_tb
    // 1. Inputs as reg, Outputs as wire
    reg clk;
    reg rst_n;
    reg start;
    reg [6:0] addr;
    reg [7:0] data;
    wire busy;
    wire scl;
    wire sda;
    reg sda_slave;
    i2c uut (
        .clk(clk),
        .rst_n(rst_n),
        .start(start),
        .addr(addr),
        .data(data),
        .busy(busy),
        .scl(scl),
        .sda(sda)
```

```
);
pullup(sda);
pullup(scl);
assign sda = sda_slave;
initial clk = 1'b0;
always #5 clk = ~clk;
initial begin
    sda_slave = 1'bz;
    repeat(8) @(posedge scl);
    @(negedge scl); // 9th SCL falling edge
    sda_slave = 1'b0; // Pull LOW = ACK
    $display("ACK sent after Address at time %0t", $time);
    @(negedge scl);
    sda_slave = 1'bz; // Release SDA
    repeat(8) @(posedge scl);
    @(negedge scl); // 9th SCL falling edge
    sda_slave = 1'b0; // Pull LOW = ACK
    $display("ACK sent after Data at time %0t", $time);
    @(negedge scl);
    sda_slave = 1'bz; // Release SDA
end
initial begin
    $fsdbDumpvars();
    // Reset
    rst_n = 1'b0;
    start = 1'b0;
```

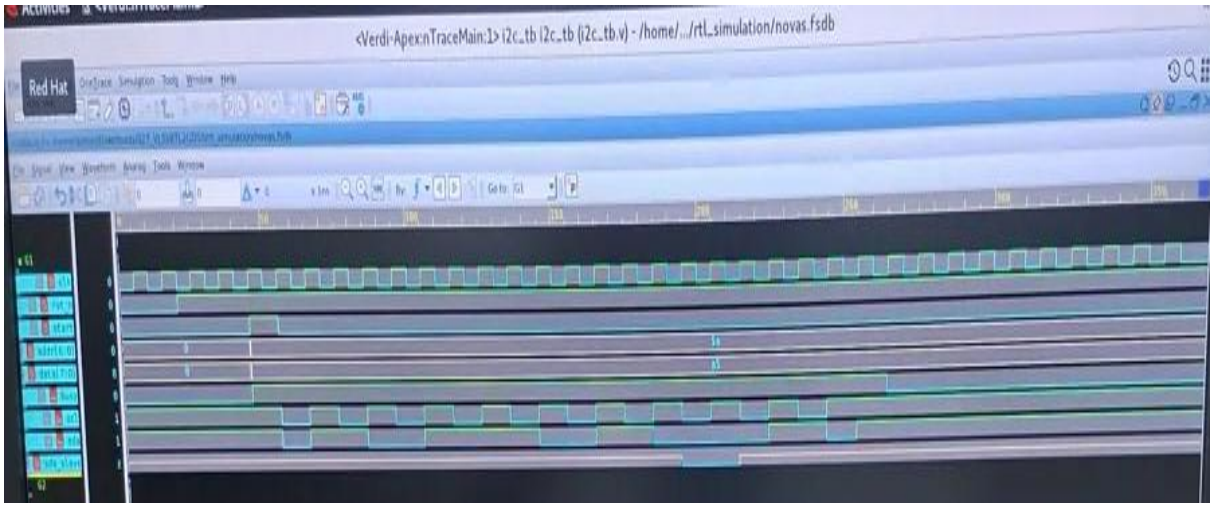
```
        addr = 7'b0;
        data = 8'b0;
        #20 rst_n = 1'b1;
        // Transaction: addr=0x5A, data=0xA5
        #20;
        @(posedge clk);
        addr = 7'h5A;
        data = 8'hA5;
        start = 1'b1;
        @(posedge clk);
        start = 1'b0;
        wait(busy == 1'b0);
        #100;
        if (uut.ack_error)
            $display(" NACK received - Transaction Failed!");
        else
            $display(" ACK received - Transaction Successful!");
        $fsdbDumpflush;
        $display("Simulation Completed Successfully.");
        $finish;
    end
endmodule
```

6. CIRCUIT DIAGRAM





7. SIMULATION RESULTS



The waveform was obtained by simulating an I2C Master RTL design using the Verdi waveform viewer. The simulation was written in Verilog HDL with a timescale of 1ns/1ns and a system clock frequency of 100MHz. The purpose of this simulation was to verify the complete functionality of the I2C protocol including the START condition, address transmission, data transmission, ACK reception, and STOP condition.

The simulation begins with a reset sequence where rst_n is held LOW for 20 nanoseconds. During this period all signals are initialized to their default values, the SCL and SDA lines remain HIGH representing the idle state of the bus, and the busy signal stays LOW. This ensures the entire system starts from a clean and known state before any transaction begins.

Once the reset is released, the testbench applies the 7-bit slave address 0x5A and the 8-bit data 0xA5 to the inputs. The start signal is then pulsed HIGH for exactly one clock cycle to trigger the I2C master. This causes the FSM to transition out of the IDLE state and the busy signal immediately goes HIGH indicating that a transaction is now in progress.

The master then generates the START condition by pulling SDA LOW while SCL remains HIGH. This is the official I2C START condition and it signals to all slave devices on the bus that a transaction is about to begin. Following the START condition, the master serially transmits the 8-bit address frame which consists of the 7-bit slave address 0x5A represented as 1011010 in binary followed by a write bit of 0. Each bit is placed on the SDA line when SCL is LOW and is sampled by the slave on the rising edge of SCL.

After transmitting all 8 bits of the address frame, the master releases the SDA line by switching it to high impedance state. At this point the slave model in the testbench pulls SDA LOW on the 9th SCL clock pulse indicating an ACK. The master samples this LOW on the rising edge of the 9th SCL pulse and confirms that a slave exists at address 0x5A and is ready to receive data. The ack_error flag remains LOW confirming successful ACK reception after the address phase.

Following the address ACK, the master proceeds to transmit the 8-bit data byte 0xA5 which is 10100101 in binary. Similar to the address phase, each bit is shifted out MSB first on the SDA line synchronized with the SCL clock. After all 8 data bits are transmitted, the master again releases SDA and the slave pulls it LOW on the 9th SCL pulse to send a second ACK. This confirms that the slave has successfully received the data byte 0xA5. The ack_error flag again remains LOW indicating no errors during the data phase.

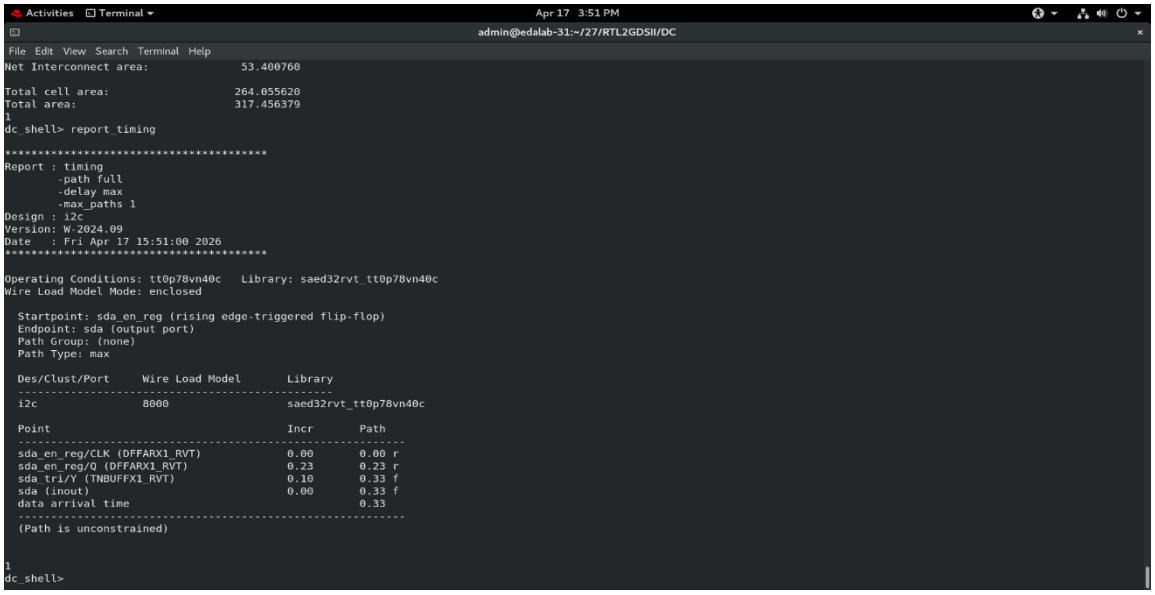
Finally the master generates the STOP condition by pulling SCL HIGH first and then allowing SDA to rise

HIGH while SCL remains HIGH. This SDA rising edge while SCL is HIGH is the unique signature of the I2C STOP condition and it officially releases the bus. At this point the busy signal goes LOW and the FSM transitions back to the IDLE state completing the entire transaction.

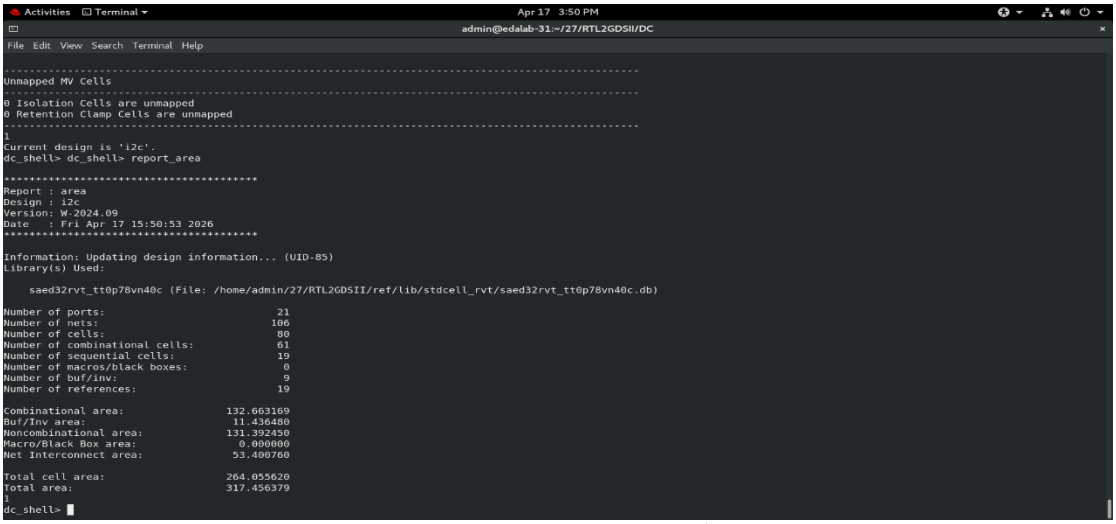
The complete FSM state transition observed during this simulation was IDLE → START → ADDR → ACK_ADDR → DATA → ACK_DATA → STOP → IDLE. All verification checks passed including correct reset behavior, proper START and STOP conditions, accurate address and data transmission, successful ACK reception after both the address and data phases, and correct deassertion of the busy signal. The ack_error signal remained LOW throughout the entire simulation confirming that the I2C master RTL design correctly implements the protocol and successfully completed an error free write transaction to slave address 0x5A with data 0xA5.

8. PERFORMANCE PARAMETERS

REPORT TIMING



REPORT AREA

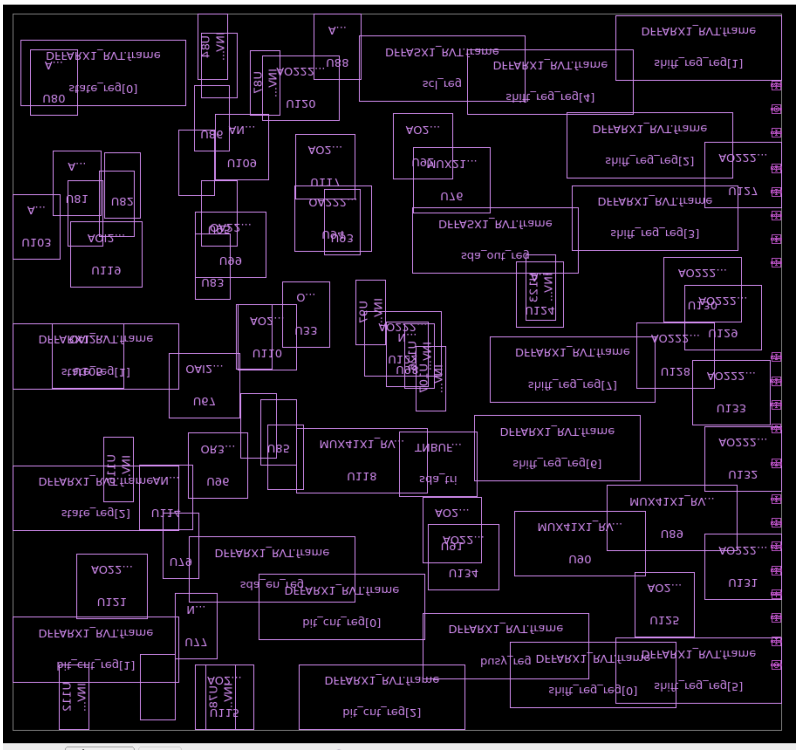


COMPILE ULTRA

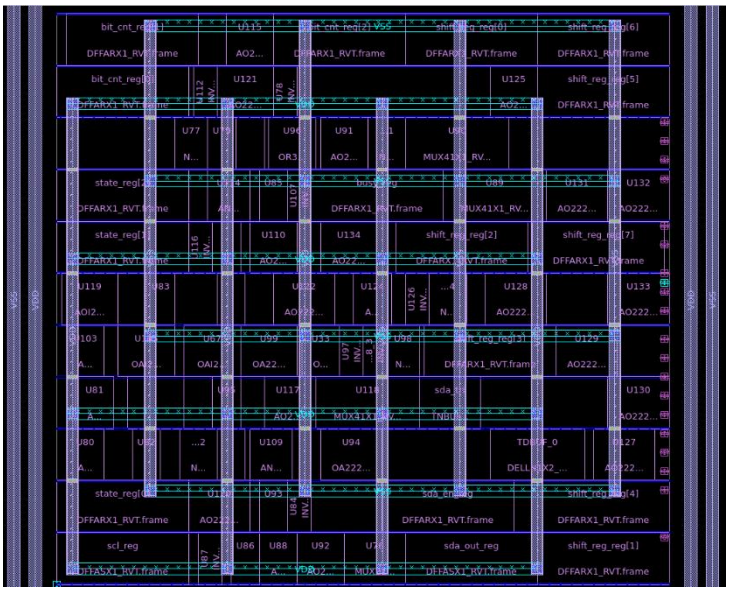
```
Activities  Terminal                    Apr 17  3:50 PM
admin@edalab-31:~/27/RTL2GDSII/DC
File  Edit  View  Search  Terminal  Help
Information: State dependent leakage is now switched from on to off.

  Beginning Pass 1 Mapping
  -----------------------
  Processing 'i2c'
CPU Load: 0%, Ram Free: 5 GB, Swap Free: 49 GB, Work Disk Free: 201 GB, Tmp Disk Free: 100 GB

  Updating timing information
Information: Updating design information... (UID-85)
Information: The library cell 'PMT3_RVT' in the library 'saed32rvt_tt0p78vn40c' is not characterized for internal power. (PWR-536)
Information: The library cell 'PMT2_RVT' in the library 'saed32rvt_tt0p78vn40c' is not characterized for internal power. (PWR-536)
Information: The library cell 'PMT1_RVT' in the library 'saed32rvt_tt0p78vn40c' is not characterized for internal power. (PWR-536)
Information: The library cell 'NMT3_RVT' in the library 'saed32rvt_tt0p78vn40c' is not characterized for internal power. (PWR-536)
Information: The library cell 'NMT2_RVT' in the library 'saed32rvt_tt0p78vn40c' is not characterized for internal power. (PWR-536)
Information: The library cell 'NMT1_RVT' in the library 'saed32rvt_tt0p78vn40c' is not characterized for internal power. (PWR-536)
Information: The target library(s) contains cell(s), other than black boxes, that are not characterized for internal power. (PWR-24)

Threshold voltage group cell usage:
>> saed32cell_svt 100.00%

  Beginning Mapping Optimizations  (Ultra High effort)
  -------------------------------
Information: There is no timing violation in design i2c. Delay-based auto_ungroup will not be performed. (OPT-780)

                              TOTAL
   ELAPSED            WORST NEG  SETUP    DESIGN                              LEAKAGE
     TIME      AREA      SLACK     COST    RULE COST         ENDPOINT            POWER
  --------- --------- --------- --------- --------- ------------------------- ---------
    0:00:01     276.5      0.00       0.0       0.0                           495424.8125
    0:00:01     273.7      0.00       0.0       0.0                           489047.7812

Threshold voltage group cell usage:
>> saed32cell_svt 100.00%

  Beginning Constant Register Removal
  -----------------------------------
    0:00:01     273.7      0.00       0.0       0.0                           489047.7812
    0:00:01     273.7      0.00       0.0       0.0                           489047.7812

Threshold voltage group cell usage:
>> saed32cell_svt 100.00%
```

ICCII

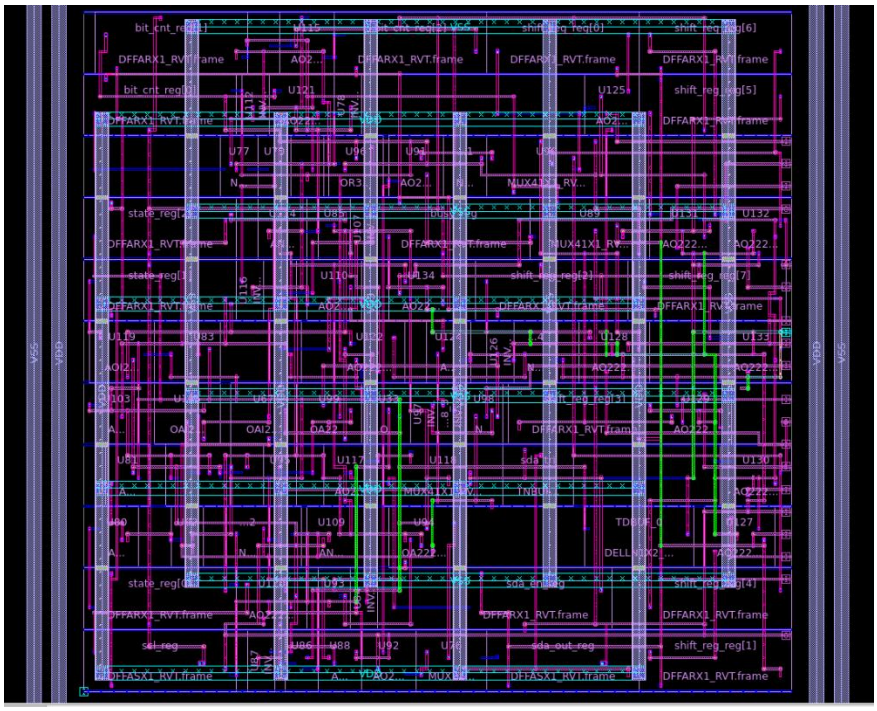
Floor Planning



PLACEMENT



CLOCK ROUTING



CONCLUSION

The project successfully demonstrates the design and implementation of an I2C Master Controller using Verilog HDL. The developed system accurately generates all essential I2C protocol operations, including START and STOP conditions, address transmission, data transfer, and ACK/NACK handling. The simulation results confirm that the master correctly communicates with the slave model, with proper synchronization using the SCL signal and reliable data transfer over the SDA line. The implementation of tri-state logic for the SDA line and the inclusion of pull-up behavior closely replicate real hardware conditions. Additionally, the ACK detection mechanism ensures robust communication by identifying successful and failed transmissions through the acknowledgment response from the slave. The FSM-based design provides a structured and efficient approach to controlling the entire communication sequence. Overall, the project validates the functional correctness of the I2C protocol at the RTL level and provides a strong foundation for further enhancements such as read operations, multi-byte transfers, and integration into larger embedded systems.

REFERENCES

1. NXP Semiconductors, "UM10204 I2C-bus specification and user manual," Rev. 6, Apr. 2014.
2. IEEE, IEEE Standard for Low-Speed Serial Communication Protocols, IEEE Publications.
3. Jan Axelson, "Serial Port Complete: COM Ports, USB Virtual COM Ports, and Ports for Embedded Systems," Lakeview Research, 2007.
4. Muhammad Ali Mazidi, "The 8051 Microcontroller and Embedded Systems," Pearson Education, 2nd Edition.
5. Synopsys, "VCS and Verdi User Guides," Synopsys Documentation.
6. Texas Instruments, "I2C Bus Pull-up Resistor Calculation Application Report," TI Literature.
7. Microchip Technology, "I2C Communication Basics and Implementation Guide," Microchip Application Notes.